

1. MAUI tarafından sağlanan layout modelleri nelerdir? Hangi tip projelerde hangi layout tipleri seçilmelidir?

- **StackLayout:** Öğeleri tek bir satır veya sütunda sırayla dizer. (Kullanım: Basit form ve liste ekranları)
- **HorizontalStackLayout:** Yalnızca yatayda öğeleri yan yana dizer. (Kullanım: Araç çubukları, yatay buton grupları)
- **VerticalStackLayout:** Yalnızca dikeyde öğeleri üst üste dizer. (Kullanım: Profil sayfaları, dikey bilgi akışı)
- **Grid:** Satır ve sütun tanımlayarak hücre bazlı düzen sunar. (Kullanım: Etiket–alan formları, dashboard kart yerleşimleri)
- **FlexLayout:** Öğeleri sarmalı ve hizalı şekilde esnek olarak düzenler. (Kullanım: Fotoğraf galerileri, dinamik etiket listeleri)
- **AbsoluteLayout:** Öğeleri X/Y koordinatlarıyla kesin konumlandırır. (Kullanım: Oyun arayüzleri, özel grafik düzenleri)
- **RelativeLayout:** Bir öğeyi başka bir öğeye veya kapsayıcıya göre oranlı konumlandırır. (Kullanım: Responsive overlap düzenleri, rozet konumlamaları)
- **ScrollView:** İçerik taşınca dikey veya yatay kaydırma sağlar. (Kullanım: Uzun metinler, çok alanlı formlar)
- **ContentView / Frame:** Tek bir alt öğeyi border, gölge veya arka planla sarar. (Kullanım: Kart bileşenleri, tekrar kullanılabilir container'lar)

2. İki sayfamız olduğunu düşünelim (MainPage.xaml ve NewPage.xaml). Bir sayfadan diğerine geçiş için “Button” kullanılıyorsa bu elemanın “XAML” ve “Button_click” olay kodlarını yazınız.

1) MainPage.xaml

xml

Kopyala Düzenle

```
<Button
    Text="Yeni Sayfaya Git"
    Clicked="Button_Clicked" />
```

2) MainPage.xaml.cs

csharp

Kopyala Düzenle

```
private async void Button_Clicked(object sender, EventArgs e)
{
    await Navigation.PushAsync(new NewPage());
}
```

3. Aşağıdaki kodlar hangi geliştirme ortamında (IDE) yazılır? Kod satırlarını tek tek açıklayınız.

```
dotnet --version
```

```
sudo dotnet workload install maui --source https://api.nuget.org/v3/index.json
```

```
dotnet new maui -n "MyMauiApp"
```

```
cd MyMauiApp dotnet build -t:Run -f net8.0-ios
```

IDE/Çalışma Ortamı:

– Bu komutlar .NET CLI (Command-Line Interface) üzerinden çalıştırılır. Genellikle Visual Studio Code veya Visual Studio 2022/2023'ün entegre terminalinde kullanılır.

Komut Açıklamaları:

1. `dotnet --version`

– Yüklü .NET SDK sürümünü ekranda gösterir.

2. `sudo dotnet workload install maui --source https://api.nuget.org/v3/index.json`

– MAUI (Multi-platform App UI) iş yükünü yükler.

3. `dotnet new maui -n "MyMauiApp"`

– "MyMauiApp" adında yeni bir .NET MAUI projesi oluşturur.

4. `cd MyMauiApp`

– Oluşturulan proje klasörüne geçer.

5. `dotnet build -t:Run -f net8.0-ios`

– Projeyi .NET 8.0 iOS hedefiyle derleyip çalıştırır.

4. Aşağıdaki XML dosyasını okuyup zip kodu "95819" olan elmanın "City" bilgisini alert yardımıyla ekrana yazdıran kodu yazınız?

```
<PurchaseOrder PurchaseOrderNumber="99503" OrderDate="1999-10-20">
  <Address Type="Shipping">
    <Name>Ellen Adams</Name>
    <Street>123 Maple Street</Street>
    <City>Mill Valley</City>
    <State>CA</State>
    <Zip>10999</Zip>
    <Country>USA</Country>
  </Address>
  <Address Type="Billing">
    <Name>Tai Yee</Name>
    <Street>8 Oak Avenue</Street>
    <City>Old Town</City>
    <State>PA</State>
    <Zip>95819</Zip>
    <Country>USA</Country>
  </Address>
  <DeliveryNotes>Please leave packages in shed by driveway.</DeliveryNotes>
  <Items>
    <Item PartNumber="872-AA">
      <ProductName>Lawnmower</ProductName>
      <Quantity>1</Quantity>
      <USPrice>148.95</USPrice>
      <Comment>Confirm this is electric</Comment>
    </Item>
    <Item PartNumber="926-AA">
      <ProductName>Baby Monitor</ProductName>
      <Quantity>2</Quantity>
      <USPrice>39.98</USPrice>
      <ShipDate>1999-05-21</ShipDate>
    </Item>
  </Items>
</PurchaseOrder>
```

```
1 XElement PurchaseOrder = XElement.Load(@"C:/Users/Marya/Desktop/deneme.xml"); // XML dosyasını belirtilen yoldan yükler
2 IEnumerable<XElement> addresses = // Address elemanlarını tutacak koleksiyonu tanımlar
3     from add in
4     PurchaseOrder.Descendants("Address") // Tüm <Address> düğümlerini alır
5     where (int)add.Element("Zip") == 95819 // Zip değeri 95819 olanları filtreler
6     select add; // Filtrelenen düğümleri seçer
7
8 foreach (var add in addresses) // Bulunan her bir Address için döner
9 {
10     String city = (String)add.Element("City"); // <City> düğümündeki metni alır
11     MessageBox.Show($"{city}", "CITY"); // Alınan şehri uyarı penceresinde gösterir
12 }
13
```

5. Herhangi bir XAML sayfası yüklendiğinde özelliklerini sizin belirlediğiniz bir “Button” yaratıp ilgili sayfadaki tüm elemanları silerek sadece “Button” elemanının görünmesini sağlayan kodları yazınız.

```
public partial class MainPage : ContentPage
{
    public MainPage()
    {
        InitializeComponent();
        // Mevcut XAML içeriğini tümüyle kaldır
        // ve kendi özelliklerinle bir Button yaratıp ata
        this.Content = new Button
        {
            Text = "Tek Buton",
            FontSize = 20,
            HorizontalOptions = LayoutOptions.Center,
            VerticalOptions = LayoutOptions.Center
        };
    }
}
```

6. “Field” ile “Variable” arasındaki farkı MAUI açısından açıklayınız. Bir “Label” için text bölümünde görülməsi gereken yazıyı belirleyen “name” isimli değişken için gerekli XAML ve .cs kodlarını yazınız.

Field ile Variable Arasındaki Fark

- **Field:** Sınıfın en üst seviyesinde (`class` bloğu içinde) tanımlanır, tüm metotlar tarafından erişilebilir ve yaşam süresi nesneyle sınırlıdır.
- **Variable:** Bir metot veya blok içinde tanımlanır, sadece tanımlandığı kapsamda (scope) geçerlidir ve blok sonlandığında bellekten kaldırılır.

XAML

xml

Kopyala Düzenle

```
<Label x:Name="lblName" Text="" />
```

MainPage.xaml.cs

csharp

Kopyala Düzenle

```
public partial class MainPage : ContentPage
{
    private string name = "Merhaba"; // field: sınıf seviyesinde tanımlanan metin

    public MainPage()
    {
        InitializeComponent();
        lblName.Text = name; // label'in Text özelliğine field ataması
    }
}
```

7. Bir “string” değişkeni içine yazılan XAML sayfası tasarımı olduğunu düşünün ve değişkenin ismi pageXAML olsun. Bu değişkenin içeriğini kullanarak yeni bir XAML sayfasını kod yardımıyla yaratıp bu sayfaya yönlennmeyi sağlayan kodları yazınız. (pageXAML strin içeriği hazır)

csharp

Kopyala Düzenle

```
// pageXAML: içinde XAML tanımı bulunan string
var newPage = (ContentPage)XamlReader.Load(pageXAML); // String'den ContentPage oluştur
await Navigation.PushAsync(newPage); // Yeni sayfaya yönlenn
```

8. İçinde iki adet eleman(herhangi) olacak şekilde dinamik bir “Frame” oluşturan kodları yazınız.

csharp

Kopyala Düzenle

```
// Dinamik Frame oluşturma ve içine iki eleman ekleme
var frame = new Frame
{
    Padding = 10,
    BorderColor = Colors.Gray,
    Content = new StackLayout
    {
        Children =
        {
            new Label { Text = "Eleman 1" },
            new Button { Text = "Eleman 2" }
        }
    }
};

// Oluşturulan Frame'i parent layout'a ekle
parentLayout.Children.Add(frame);
```

9. Aşağıdaki kodun oluşturduğu şekli çiziniz

```
<Frame BorderColor="Gray"
      CornerRadius="5"
      Padding="8">
  <StackLayout>
    <Label Text="Card Example"
          FontSize="14"
          FontAttributes="Bold" />
    <BoxView Color="Gray"
            HeightRequest="2"
            HorizontalOptions="Fill" />
    <Label Text="Frames can wrap more complex layouts to create more complex UI
components, such as this card!"/>
  </StackLayout>
</Frame>
```

Card Example

Frames can wrap more complex layouts to create more complex UI components, such as this card!

kotlin

Kopyala Düzenle

Card Example

Frames can wrap more complex layouts to create more complex UI components, such as this card!

- ← Başlık (FontSize=14, Bold)
- ← İnce gri çizgi (Height=2, Fill)
- ← Açıklama metni (Label)

- **CornerRadius=5:** Köşeler hafif yuvarlatılmış
- **BorderColor=Gray:** Gri kenarlık
- **Padding=8:** Çerçevenin iç kenar boşluğu
- **StackLayout içinde (dikey):**
 1. "Card Example" (14 pt, kalın)
 2. Gri yatay çizgi (2 birim yükseklik)
 3. Alt metin (çok satırlı Label)

10. MAUI üzerinde Google Map kullanmak için gerekli olan adımları yazınız.

1. API Anahtarı Oluşturma

Google Cloud Console'da hem Android hem de iOS için "Maps SDK" hizmetlerini aktif hâle getirip, her platform için ayrı API anahtarları alın.

2. NuGet Paketi Ekleme

Projenin NuGet paket yöneticisinden **Microsoft.Maui.Controls.Maps** paketini yükleyerek harita bileşenlerine erişimi sağlayın.

3. Harita Desteğini Aktifleştirme

Uygulama başlatma ayarlarında (MauiProgram.cs veya benzeri yapılandırmada) harita servisini devreye geçirerek MAUI'ye harita modülünü tanıtır.

4. Android Ayarları

Android projesinin manifest dosyasında ilgili meta-data girdisini oluşturarak Android API anahtarınızı tanımlayın.

5. iOS Ayarları

iOS projesinin Info.plist dosyasına **GMSApiKey** girdisi ekleyerek iOS API anahtarınızı yapılandırın.

6. XAML Tanımlaması

XAML sayfanıza Maps ad alanını ekleyip **<Map>** kontrolünü yerleştirerek kullanıcı arayüzünde harita görünümünü oluşturun.

7. Başlangıç Konumu Ayarı

Kod-behind veya ilgili yapıdaki ayarlarda haritanın başlangıç görüntüsünde hangi şehir/koordinat bölgesinin görüneceğini belirleyin.

11. Bir "ImageButton" için gerekli XAML kodlarını yazınız (Source, Clicked, WidthRequest ad HeightRequest, image1.jpg). Resme tıklandığında alert'e "resim1" isimli "Label" elemanın Text kısmına "MAUI is not everything!" yazan .cs kodlarını yazınız.

XAML

```
xml
Kopyala Düzenle

<Label x:Name="resim1" Text="" />
<ImageButton
    Source="image1.jpg"
    Clicked="OnImageButtonClicked"
    WidthRequest="100"
    HeightRequest="100" />
```

MainPage.xaml.cs

```
csharp
Kopyala Düzenle

private async void OnImageButtonClicked(object sender, EventArgs e)
{
    resim1.Text = "MAUI is not everything!"; // Label'in Text'ini ayarla
    await DisplayAlert("Uyarı", resim1.Text, "OK"); // Alert ile göster
}
```

12. Çizim için elimizde “canvas” isimli bir ICanvas değişkeni mevcut. Bunu değişkeni kullanarak bir “Gradient” tipi de vererek “rounded” rectangle çizdiren .cs kodlarını yazınız.

csharp

Kopyala Düzenle

```
using Microsoft.Maui.Graphics; // ICanvas, LinearGradientPaint, RectF, Colors için

// 1. Gradient boyası oluştur
var gradient = new LinearGradientPaint
{
    StartColor = Colors.Red,
    EndColor    = Colors.Blue,
    StartPoint  = new Point(0, 0),
    EndPoint    = new Point(100, 0)
};

// 2. Gradient'i doldurma boyası olarak ayarla
canvas.SetFillPaint(gradient, new RectF(10, 10, 100, 50));

// 3. Yuvarlak köşeli dikdörtgeni çiz
canvas.FillRoundedRectangle(10, 10, 100, 50, 10);
```